

Seamless Transition Between Gaussian Splatting and Mesh Rendering

Jerneja Krajcar

University of Ljubljana

Abstract

This seminar presents a prototype for distance-dependent transitions between mesh rendering and 3D Gaussian Splatting. The goal is to use a mesh representation for distant, low-detail views and a Gaussian representation for close-up views, while avoiding abrupt visual changes. The current main path uses Mesh2Splat exports as the Gaussian input, builds levels of detail from those exported PLY files, computes smooth transition weights from camera distance, and exposes the transition in a web viewer.

Keywords: 3D Gaussian Splatting, mesh rendering, level of detail, hybrid rendering, smooth transitions

1. Introduction

Mesheres are compact, widely supported, and reliable for low-detail views. 3D Gaussian Splatting (3DGS), as introduced by Kerbl et al. [KKLD23], is useful for high-quality appearance because it represents a scene with many semi-transparent Gaussian primitives instead of only triangle surfaces. The seminar task is therefore a hybrid rendering problem: use the scalable behaviour of meshes at distance, but switch to richer Gaussian detail when the camera moves closer.

The main difficulty is not only selecting the right representation, but changing between representations without popping, brightness jumps, or visible discontinuities. The current implementation is a prototype, not a production renderer.

2. Problem

The required system should support a seamless transition between two scene representations:

- a simplified mesh, suitable for low-detail or far-away views;
- one or more Gaussian splat representations, suitable for higher-detail close-up views.

A naive switch at a fixed distance produces visible popping because the two renderers do not necessarily match in color, coverage, silhouette, or sampling density. A better solution must make the representation change gradual and should keep neighbouring Gaussian levels spatially related, so that a denser level of detail does not look like a completely unrelated point set.

3. Related Work

3DGS provides the basic representation used for the Gaussian side of this project [KKLD23]. SuGaR aligns Gaussians with surfaces and uses this relationship for mesh extraction and high-quality mesh rendering [GL24]. This is directly relevant because it shows that mesh and Gaussian representations can be connected through surface alignment rather than treated as unrelated assets.

SplattingAvatar embeds Gaussian splats on a mesh for real-time human avatar rendering [SWL*24]. Its main target is animatable humans, but the idea of attaching or aligning Gaussian appearance to mesh structure is related to this seminar work.

Mesh2Splat [Ele25] is used as the main practical conversion path because it offers a direct way to create Gaussian splat PLY files from mesh assets. The repository also includes an experimental NVIDIA/CUDA `gsplat` training path, but this is kept separate from the Mesh2Splat workflow.

4. Implemented Solution

The current prototype is centred on a Mesh2Splat workflow.

In the main workflow, a mesh is converted with Mesh2Splat and several Gaussian PLY files are exported with different splat counts. These files are placed under `data/mesh2splats/`. The viewer groups them by mesh name and uses the splat count in the filename as the LOD key.

The web viewer then loads the source mesh and the matching Mesh2Splat LOD set. It can also use an initialized preview built from mesh sampling, but this is now mainly a fallback for development and testing rather than the intended final result.

An experimental training script was added for NVIDIA/CUDA

machines. It renders synthetic views from a mesh, exports a COLMAP-style dataset, prepares a `gsplat` training command, can run the trainer when `-run-trainer` is provided, and stores the result under `data/trained_gaussians/`. A separate environment check reports whether CUDA-enabled PyTorch and the expected `gsplat` trainer script are available.

4.1. Transition Model

The transition is controlled by camera distance. A smoothstep function is used for all fades, so weights change gradually instead of linearly snapping at a threshold. The mesh weight fades out between configurable far and near distances. The remaining weight budget is assigned to active Gaussian LOD bands and then normalized, so that the combined mesh and Gaussian contribution stays stable.

For the default configuration, the mesh starts fading near distance 3.6 and is mostly replaced by Gaussian rendering near distance 2.2 in normalized object units. Gaussian LOD bands then progressively select counts such as 10, 100, 500, 5000, and 20000 splats as the camera moves closer.

4.2. Blending

The current blend is an image-space weighted sum:

$$I = w_m I_m + \sum_i w_i I_i,$$

where I_m is the mesh render, I_i are Gaussian LOD renders, and the weights sum to one. This keeps the transition simple and inspectable. It does not yet solve deeper alignment problems such as exact silhouette matching, depth-aware compositing between representations, or learned color correction.

5. Alternatives Considered

Hard LOD switching would be simpler, but it directly causes popping and therefore does not satisfy the solution of the problem.

Depth-aware screen-space blending could reduce artifacts where the mesh and Gaussians overlap incorrectly. It is a good future direction, but the first version uses color blending because it is easier to debug and works with the current lightweight renderers.

Direct CUDA/gsplat training was added as an experimental route for generating splats from synthetic training views on an NVIDIA/CUDA machine. It was not kept as the main path because it depends on a heavier CUDA, PyTorch, compiler, and training setup.

Full surface-aligned Gaussian optimization in the style of SuGaR would likely improve geometric consistency, but it is heavier than the current seminar prototype and would require a stronger training and reconstruction pipeline.

6. Results

The prototype currently focuses on the interactive viewer. It loads a source mesh together with Mesh2Splat-exported Gaussian LOD

files and displays a continuous transition controlled by camera distance.

In the current workflow, the exported Mesh2Splat files can contain several LOD sizes, for example a small far-distance splat set and progressively denser near-distance splat sets. The viewer switches between these LOD files proportionally rather than with a single hard switch, so the transition can be inspected with the pipeline slider.

The CPU/offline renderer remains useful for validation, but it is no longer presented as the main route for high-quality Gaussian generation. The added NVIDIA/CUDA training path can produce trained Gaussian PLY files for the viewer, while GPU rasterization is still outside the current implementation.

7. Limitations and Future Work

The largest current limitation is that the practical LOD workflow still depends on externally exported Mesh2Splat PLY files. The CUDA/gsplat route adds dataset export and trainer execution support, but it is experimental and depends on an external NVIDIA training environment.

Future work should focus on four improvements: depth-aware blending, better alignment between mesh and Gaussian geometry, quantitative comparison on real assets, and a more stable CUDA/gsplat workflow. A later version could also bind Gaussian parameters more directly to mesh surfaces, following the direction suggested by SuGaR and SplattingAvatar.

8. Conclusion

The implemented prototype demonstrates a clear first step toward seamless mesh-to-Gaussian LOD transitions. It does not claim to solve all visual artifacts, but it establishes the key structure: mesh rendering, Mesh2Splat Gaussian LODs, distance-based smooth weights, interactive inspection, and an experimental NVIDIA/CUDA training route for trained splats.

References

- [Ele25] ELECTRONIC ARTS: Mesh2Splat: Fast mesh to 3d gaussian splat conversion. <https://github.com/electronicarts/mesh2splat>, 2025. 1
- [GL24] GUÉDON A., LEPETIT V.: SuGaR: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/html/Guedon_SuGaR_Surface-Aligned-Gaussian-Splatting_for_Efficient_3D_Mesh_Reconstruction_and_CVPR_2024_paper.html. 1
- [KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics* 42, 4 (2023). doi:10.1145/3592433. 1
- [SWL*24] SHAO Z., WANG Z., LI Z., WANG D., LIN X., ZHANG Y., FAN M., WANG Z.: Splattingavatar: Realistic real-time human avatars with mesh-embedded gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: <https://openaccess.thecvf.com/content/CVPR2024/papers/>

[Shao_SplattingAvatar_Realistic_Real-Time_Human_Avatars_with_Mesh-Embedded_Gaussian_Splatting_CVPR_2024_paper.pdf](#). 1